



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería de Software

ANÁLISIS DE CASO: SISTEMAS DISTRIBUIDOS

“Fallo de Amazon Web Services us-east-1 (Abril 2011)”

Integrantes:

Bedón Viteri Keyla Betzabe
Castro López Pedro Leonardo
Emanuel Pino Juliana Romina
Vinuela Sánchez Harold Nicolás

Asignatura:

Aplicaciones Distribuidas

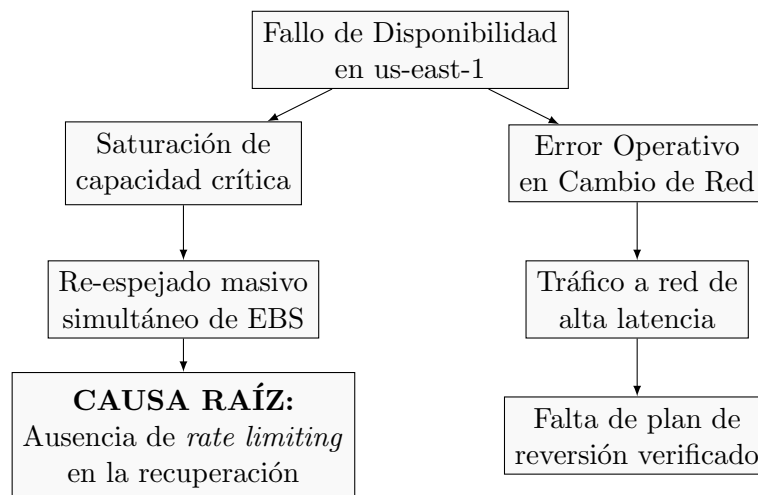
Quevedo, Ecuador
19 de mayo de 2026

1. Descripción del Incidente

El 21 de abril de 2011, la región *us-east-1* de AWS sufrió una interrupción de 96 horas originada por un error humano durante un cambio de red rutinario [2, 3]. El tráfico fue desviado erróneamente a una red de alta latencia, lo que provocó que los volúmenes de Elastic Block Store (EBS) perdieran sus réplicas e iniciaran procesos de re-espejado masivos [3]. La demanda simultánea saturó la capacidad de almacenamiento, generando una “tormenta de re-espejado” autoamplificada que dejó al sistema sin disponibilidad para priorizar la consistencia de los datos (modelo CP) [4, 5].

2. Identificación de Causas Raíz - Árbol de Fallos

Mediante el análisis de los “5 Porqués”, se determinó la siguiente estructura de fallos [6, 7]:



3. Impacto Cuantificado

El impacto del incidente fue severo y se resume en las siguientes métricas [8]:

- **Disponibilidad:** 96 horas de interrupción crítica del servicio.
- **Instancias EC2:** 1.87 % de las instancias en la región quedaron inaccesibles permanentemente.
- **Pérdida de Datos:** 0.07 % de los volúmenes EBS sufrieron pérdida irrecuperable.
- **Ecosistema:** Caída total de servicios como Reddit, Foursquare y Quora.

4. Estrategias de Mitigación y Lecciones Aprendidas

AWS y la industria adoptaron las siguientes estrategias tras el análisis post-mortem [7, 9]:

1. **Rate Limiting y Backoff:** Implementación de límites de velocidad en procesos de recuperación para evitar saturaciones en cascada.
2. **Chaos Engineering:** Adopción de pruebas de inyección de fallos para validar la resiliencia antes de eventos reales.

3. **Diseño Multi-AZ:** Obligatoriedad de arquitecturas que no dependan de una única zona de disponibilidad.

5. Aplicación al Proyecto Fin de Curso (PFC)

Como equipo, aplicaremos las siguientes decisiones basadas en el caso [10, 11]:

- **Definición CAP:** Documentar si el PFC prioriza consistencia o disponibilidad en el ADR.
- **Circuit Breaker:** Implementar el patrón para evitar fallos en cascada entre micro-servicios.
- **Resiliencia:** Realizar pruebas deteniendo contenedores Docker en la entrega final.

Referencias

- [1] Amazon Web Services, “Summary of the Amazon EC2 and Amazon RDS Service Disruption in the US East Region,” *AWS Service Health Dashboard*, Abr. 2011. [Online]. Disponible: <https://aws.amazon.com/message/65648/>
- [2] E. Brewer, “Towards robust distributed systems,” en *Proc. 19th ACM Symp. Principles Distributed Computing (PODC)*, Portland, OR, USA, Jul. 2000, p. 7. doi: 10.1145/343477.343502
- [3] S. Gilbert y N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51-59, Jun. 2002. doi: 10.1145/564585.564601
- [4] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4ta ed. Maastricht, Países Bajos: Maarten van Steen, Ene. 2025, cap. 8 “Fault Tolerance”.
- [5] F. Cristian, “Understanding fault-tolerant distributed systems,” *Commun. ACM*, vol. 34, no. 2, pp. 56-78, Feb. 1991. doi: 10.1145/102792.102801
- [6] R. Chahal, “The AWS US-East-1 outage: A Postmortem Analysis,” *IEEE Cloud Computing*, vol. 1, no. 2, pp. 12-18, Jun. 2014.